STRUCTML HW 1

Zina Assi

213813165

Zina.assi@campus.technion.ac.il

Abed-al-rahman Badran

325424752

badran.abed@campus.technion.ac.il

Q1: Suggest and implement 3 join-aggregate features that you think can help in the prediction task. What are they? How do you hypothesize that they are related to the prediction task?

We picked three features, each pulling from a different table in the database to try to capture different sides of "is this user active?".

Feature 1: num_posts_before_t - number of posts the user has made before the target timestamp. Counts rows in the posts table where OwnerUserId matches the user and CreationDate <= timestamp. We expect users who post a lot to keep posting in the next 3 months, so this should be a positive predictor of the label.

Feature 2: num_votes_before_t - number of votes the user cast before the timestamp. Counts rows in the votes table where UserId matches and CreationDate <= timestamp. Voting is the easiest form of participation on Stack Overflow, so even less active users might vote, which should make this a broad activity proxy. But a lot of vote rows have UserId = <NA> in the dataset because votes are anonymized, so the feature ends up very sparse .

Feature 3: days_since_last_comment - how many days have passed since the user's last comment before the timestamp. NaN if they never commented. Recency feels like it should matter a lot for this task: someone who commented yesterday is much more likely to engage soon than someone whose last comment was 4 years ago. The NaN bucket itself encodes "never engaged with comments".

Q2 :

2. Implement an automatic join-aggregate feature generation algorithm:
    a. Given an input parameter `max_depth`, perform all joins of up to `max_depth` hops. For example, a join path with a single hop: target ⋈ users. A join path with 2 hops: target ⋈users⋈ posts.
    [Edited:] For joining the target table to the others, use left join to keep all rows of the target table. For all other joins, use a natural join. For more information on types of join see here:
    https://www.atlassian.com/data/sql/sql-join-types-explained-visually.
    b. For each joined table achieved this way, for each of the columns in the joined table, create aggregated features using aggregations that match its type.
        i. For numeric columns, use mean, count and sum.
        ii. For categorical/text columns, use count.
        iii. For timestamp columns, use min and max.
    c. Run this algorithm with `max_depth`=2. Collect the original target table and the new features to a single `pandas` dataframe (we will refer to it as `feature_matrix` for the rest of the exercise).

Q3:
How many new features would be created for max_depth in {1,2,3}? Explain your answer.

max_depth	Number of new features
1	16
2	51
3	111

For each path ending in leaf table T,

the leaf contributes (3 × num_numeric_cols) + (1 × num_text_cols) + (2 × num_timestamp_cols) aggregated features.

Per-table aggregation contributions (after skipping PK + FK):

- users → 9
- (DisplayName 1, Location 1, ProfileImageUrl 3, WebsiteUrl 1, AboutMe 1, CreationDate 2)
- posts → 10,
- postHistory → 10,
- votes → 5,
- badges → 5
- (Class and TagBased treated as categorical per schema),
- comments → 5,
- postLinks → 5

At depth 1 we have target ⋈ users, contributing 7 raw users columns (kept from the LEFT JOIN, AccountId dropped) plus 9 aggregations = **16**.

At depth 2 we add 5 paths (users ⋈ posts, postHistory, votes, badges, comments) contributing 10+10+5+5+5 = 35 aggregations → 16 + 35 = **51**.

At depth 3 we add 8 more length-2 paths from users; their leaf contributions sum to 60 → 51 + 60 = **111**.

Q4

fm_train.shape = (1,360,850, 54).

The original target table was (1,360,850, 3) just OwnerUserId, timestamp, and contribution.

Our feature matrix has the same number of rows (because we used a LEFT JOIN to the target, so no rows are dropped), plus 7 raw users columns + 44 aggregated features = 51 new columns. So 3 + 51 = 54 columns total.

One thing worth noting: some of the intermediate joins were huge, but the groupby step collapses everything back down to one row per (user, timestamp) target row, so the output stays at 1.36M rows.

Q5: Give examples of 3 of the newly extracted features (that were not part of the original table), and explain their meaning (what they represent).

1. posts__CreationDate__max: the timestamp of the user's most recent post (at or before the target timestamp). This captures how recently the user has posted. NaN if the user never posted.
2. posts__PostTypeId__mean: average post type for the user's posts. PostTypeId is 1 for questions and 2 for answers, so this captures whether the user tends to ask questions or answer them.
3. comments__Text__count: the number of non-null Text fields in the user's comments, which is basically just the number of comments they have authored.

Q6 : Are there any missing values in the extracted features, and why?

Yes, lots of missing values. They come from a few different places:

1. **No matching rows after the join + temporal cutoff.** If a user has never commented before the timestamp, the aggregations over comments are NaN. (We explicitly set the count aggregations to 0 since "no rows" is meaningfully zero count, but mean/sum/min/max stay NaN.)
2. **NULL values in the source data.** votes.UserId is mostly NULL in this dataset because votes are anonymized. Several users columns (like ProfileImageUrl, Location) are also sparse.
3. **Aggregating over all-NaN inputs** just gives NaN back.

We think the missingness pattern itself is signal here - "this user has never voted before this timestamp" tells us something. Tree-based models (DT, XGBoost) handle NaN natively. For LogReg and the NN we needed to impute (we used median for the numeric features).

Describe the architecture and training parameters in your report :

Architecture:

A small fully-connected feed-forward network, defined as a torch.nn.Sequential:

Input (51 features, standardized + median-imputed)

↓

Linear(51 → 256) → ReLU → Dropout(p=0.3)

↓

Linear(256 → 128) → ReLU → Dropout(p=0.3)

↓

Linear(128 → 64) → ReLU → Dropout(p=0.3)

↓

Linear(64 → 1) (raw logit; we apply sigmoid at evaluation time)

Training parameters:

Parameter	**Value**	**Reason**
Loss	BCEWithLogitsLoss(pos_weight=~19)	Binary classification; pos_weight handles the 5% positive-rate imbalance so the model doesn't ignore the minority class.
Optimizer	Adam	Reliable default; no need for learning-rate schedules at this scale.
Learning rate	1e-3	Standard Adam starting point. Val AUC was stable.
Batch size	4096	Large enough to amortize Python overhead on 1.36M rows; small enough to fit comfortably in CPU memory.
Epochs	4	Val AUC plateaued around epoch 3-4; more epochs started to drift without improving.
Activation	ReLU	Standard for hidden layers; cheap and works well with standardized inputs.
Dropout	0.3	Regularizes the wider 256/128 layers; without it the network started memorizing the training set.

Weight init	PyTorch default (Kaiming uniform for Linear)	No custom init needed for this depth.
Random seed	0 (torch.manual_seed)	Reproducibility.

Q7:

Main results (best variant per model, train/val):

Model	Accuracy	ROC AUC	Precision	Recall	F1
Decision Tree	**0.769 / 0.670**	**0.808 / 0.861**	**0.141 / 0.070**	**0.708 / 0.880**	**0.235 / 0.130**
XGBoost (best)	**0.790 / 0.735**	**0.846 / 0.879**	**0.158 / 0.083**	**0.736 / 0.852**	**0.260 / 0.153**
Logistic Regression	**0.778 / 0.718**	**0.798 / 0.839**	**0.140 / 0.075**	**0.669 / 0.802**	**0.232 / 0.138**
Custom NN (MLP)	**0.785 / 0.733**	**0.814 / 0.854**	**0.146 / 0.079**	**0.682 / 0.800**	**0.241 / 0.144**

Variants table:

Model	Variant	Train AUC	Val AUC
DT	**depth=5**	**0.808**	**0.861**
DT	**depth=10**	**0.843**	**0.839**
DT	**depth=20**	**0.945**	**0.606**
DT	**depth=None, min_leaf=50**	**0.926**	**0.840**
XGB	**depth=4, n=200, lr=0.10**	**0.846**	**0.879**
XGB	**depth=6, n=200, lr=0.10**	**0.861**	**0.877**
XGB	**depth=8, n=400, lr=0.05**	**0.885**	**0.867**
LR	**C=0.1**	**0.798**	**0.839**
LR	**C=1.0**	**0.801**	**0.830**
LR	**C=10.0**	**0.801**	**0.828**
MLP	**(128, 64), do=0.2**	**0.814**	**0.854**
MLP	**(256, 128, 64), do=0.3**	**0.814**	**0.851**

Hyperparameter search - what we changed and what values we tried:

Model	Hyperparameter(s)	Values tried	Why these
Decision Tree	max_depth, min_samples_leaf	depth ∈ {5, 10, 20, None}; with None we set: min_samples_leaf=50	Shallow trees underfit, very deep ones overfit (we saw it cleanly: depth=20 → train AUC 0.945, val AUC 0.606). Sweeping depth covers the bias/variance tradeoff.
XGBoost	max_depth, n_estimators, learning_rate	(4, 200, 0.10), (6, 200, 0.10), (8, 400, 0.05)	Standard boosting tradeoff: shallower trees with more iterations vs. deeper trees fewer iterations. Smaller LR + more iterations is a more conservative fit.
Logistic Regression	C (inverse L2 strength)	{0.1, 1.0, 10.0}	Sweeps from strong regularization (0.1) to almost none (10.0). With 51 features and >1M rows, regularization barely matters here, which the results confirm.
Custom NN	architecture, dropout	(128,64) dropout=0.2 and (256,128,64) dropout=0.3	Tests whether a wider/deeper network with more dropout buys anything. Marginal win for the deeper one.

All variants were re-selected by **validation ROC-AUC**.

Q8 : Explain the meaning of each evaluation measure (accuracy, AUROC, precision, recall, F1). Explain the tradeoffs between the measures. Which ones do you think will be most helpful in our prediction task, considering the balance between classes?

What each metric means and which ones matter most

1. **Accuracy**: fraction of predictions that are correct overall ((TP+TN)/N).
2. **ROC AUC**: probability that a random positive example gets a higher score than a random negative example. It's threshold-independent.
3. **Precision**: out of the users we predicted will engage, how many actually did (TP / (TP+FP)).
4. **Recall**: out of the users who actually engaged, how many we caught (TP / (TP+FN)).
5. **F1**: harmonic means of precision and recall (so it balances them, at the chosen threshold).

Tradeoffs between them:

Accuracy is misleading when classes are imbalanced. With 5% positives, you could just predict "0" for everyone and get 95% accuracy with 0 useful information.

Precision and recall trade off against each other. Lowering the decision threshold catches more positives but also calls more negatives positive by mistake. F1 is one point on this tradeoff curve.

ROC AUC doesn't depend on the chosen threshold, which makes it the most reliable metric for comparing models, especially when classes are imbalanced.

we think ROC AUC matters the most because:

It's robust to class imbalance,It's the same metric the relbench paper uses, so we can compare results and It doesn't force us to pick a deployment threshold up front.

Accuracy is the least useful here because of the imbalance. F1, precision and recall give a more concrete picture at a specific threshold (we used 0.5), but we mostly trust AUC for the head-to-head model comparison.

Q9: Discuss the results. Which models were better on the training set? And on the validation set? What might be the reasons for these advantages, based on the mechanisms behind each model, and each evaluation measure? How does the choice of the schema and learning task affect the results?

Train vs. validation rankings

Best-variant table (the one we picked, by val AUC):

Model	**Train AUC**	**Val AUC**
XGBoost	0.846	0.879
Decision Tree	0.808	0.861

Model	Train AUC	Val AUC
Custom NN (MLP)	0.814	0.854
Logistic Regression	0.798	0.839

All four models comfortably beat the 0.65 baseline from the relbench paper for users-only features.

One thing we noticed: val AUC is higher than train AUC for XGBoost, MLP, and the shallow DT. We think this is because the val set is later in time and has a lower positive rate (3% vs 5%), so the engaged users in val are mostly recently active accounts, which are easier to score.

Why XGBoost won

Our features are almost all counts, sums, and timestamps with a lot of NaN. That's exactly what gradient boosted trees handle best:

1. NaN is treated natively, so "user did nothing" stays a meaningful signal.
2. Boosting captures non-linear interactions (many trees on residuals).
3. Shrinkage + depth caps prevent overfitting, which is why depth=4 with 200 trees beat the deeper variants on val.

Why the MLP was close second

It got 0.871. With standardized features and dropout, it can capture similar non-linearities to XGBoost. It loses a little because:

1. median imputation throws away the "missing" signal.
2. every input feed every neuron so noisy features add gradient noise - XGBoost just ignores them.

Why DT lags behind XGBoost

A single tree is high variance. Once it splits at the root, that split dominates (we saw **70%** of importance on one feature). Going deeper overfits (depth=20 → val **0.606**), staying shallow underfits. XGBoost averaging many shallow trees fixes exactly this problem.

Why LR was last

LogReg is linear, but most of our signal isn't:

1. Recency features are non-monotonic in calendar time - both very old and very new timestamps can mean "not engaged".
2. Counts saturate (0→10 posts matter a lot, 100→1000 barely does).
3. LR can't model interactions like "posts matter more when AboutMe is filled in".

Why the metrics tell different stories

Accuracy is similar across all models (0.72-0.79). With 95% negatives, accuracy is dragged toward whatever our class-weighted threshold produces, so it doesn't separate the models meaningfully.

ROC-AUC cleanly separates the models (0.824 to 0.882) because it measures ranking quality across all thresholds.

Precision/recall/F1 look bad (precision 0.08-0.16) but that's an artifact of using threshold 0.5 with class weighting. They reflect our threshold choice, not model quality.

AUC is the right primary metric here.

How the schema affects results

The target only foreign-keys into users, so every feature is a user-history aggregate. The tables with the most rows per user (postHistory, comments, badges) dominate the importances. postLinks isn't reachable at depth 2, so it contributes nothing - depth 3 would surface that signal.

How the task affects results

Predicting next-3-months engagement is churn prediction, and churn is driven by recency. That's why recency features (comments__CreationDate__max, posts__CreationDate__max, badges__Date__max) are top-10 for the tree models, and why LR - which can't fit non-monotonic curves - does worse. The 3-5% positive rate makes accuracy useless and ROC-AUC the natural fit, which is also what the relbench paper uses.

Q10:

How feature importance is computed in each model

Decision Tree (sklearn). For every split in the tree that uses feature j, sum up the Gini-impurity reduction at that split, weighted by the number of training samples reaching the node. Add this up for all splits using j, then normalize so the importances over all features sum to 1. A feature scores high if it gets used at splits that cleanly separate large groups of samples.

XGBoost. The default feature_importances_ uses **gain**: for each feature, sum the improvement in the loss function from every split using that feature, across all boosted trees, then normalize. It rewards features that consistently reduce the loss, not just features that get split on a lot. XGBoost

also has alternatives ("weight" = count of splits, "cover" = avg samples per split) accessible via get_booster().get_score(importance_type=...), but we used the default.

Logistic Regression. Sklearn's LR doesn't have feature_importances_ . It has coef_. Because we standardized our inputs (mean 0, std 1), | coef_[j] | is directly comparable across features and is a sensible importance score. A larger |coef_[j]| means a one-σ change in feature j moves the log-odds more.

Q11:

Top 10 features per model

Rank	Decision Tree	XGBoost	Logistic Regression
1	posts__Body__count (0.70)	posts__PostTypeId__count (0.52)	votes__CreationDate__min (16.17)
2	posts__PostTypeId__count (0.13)	posts__PostTypeId__sum (0.15)	votes__CreationDate__max (16.05)
3	users__AboutMe__count (0.05)	users__AboutMe__count (0.04)	postHistory__ContentLicense__count (9.75)
4	badges__Date__max (0.03)	comments__ContentLicense__count (0.04)	postHistory__UserDisplayName__count (9.73)
5	posts__CreationDate__max (0.03)	users__Location__count (0.04)	posts__Title__count (9.47)
6	comments__CreationDate__max (0.03)	postHistory__PostHistoryTypeId__sum (0.03)	posts__OwnerDisplayName__count (9.45)
7	postHistory__CreationDate__max (0.02)	postHistory__CreationDate__max (0.03)	badges__Date__max (4.07)
8	comments__Text__count (0.01)	comments__CreationDate__max (0.02)	badges__Date__min (3.66)
9	badges__Name__count (0.003)	badges__Date__max (0.02)	posts__CreationDate__max (0.77)
10	posts__ContentLicense__count (0.002)	posts__CreationDate__max (0.01)	posts__PostTypeId__count (0.73)

Commonalities

All three models pick up on user activity volume. Counts of posts, comments, postHistory, and badges show up in everyone's top 10. posts__CreationDate__max is in the top 10 of all three models, and posts__PostTypeId__count appears in both XGBoost's and LR's top 10. So the auto-generated features are surfacing the same kind of signals we manually picked in Q1, how much the user has done and how recently which makes sense.

DT is dominated by one feature. posts__Body__count alone gets 70% of the importance. Once a single tree splits on the strongest feature at the root, that split dominates the whole tree's impurity reduction. Correlated alternatives like posts__PostTypeId__count get most of the rest, and other features barely matter.

XGBoost spreads importance more evenly. Top feature is 52% and the rest of the top 10 mixes post counts, user-attribute counts (AboutMe, Location), postHistory counts, and recency timestamps. Boosting fits each tree on residuals, so once the top signal is partly explained, secondary features get to contribute.

LR is now dominated by timestamps and text counts. The top two are votes__CreationDate__min/max with coefficients around 16, then a cluster of high-cardinality text count features (postHistory__ContentLicense__count, postHistory__UserDisplayName__count, posts__Title__count, posts__OwnerDisplayName__count) at ~9. We think the reason is that after we converted datetime to seconds-since-epoch, timestamp features have a smooth continuous range that LR can fit linearly, and the high-cardinality counts have enough variance per standard deviation to dominate.

Bottom line. The same underlying signal (user activity) gets attributed differently because the three metrics measure different things: Gini reduction (DT), loss gain (XGB), log-odds shift per σ (LR). Trees pick one feature from a correlated group; LR weights all of them by their standardized variance. Divergent top-10 lists are expected, and reading across all three is more informative than any single ranking.

Part 2:

Q13: Train and compare the models again and report the performance measures in a table such as the one given in question 7.

Results (train / validation):

Model	Accuracy	ROC AUC	Precision	Recall	F1
Decision Tree	0.836 / 0.856	0.925 / 0.810	0.218 / 0.111	0.879 / 0.590	0.349 / 0.187
XGBoost	0.886 / 0.925	0.960 / 0.865	0.296 / 0.200	0.933 / 0.557	0.450 / 0.294
Logistic Regression	0.871 / 0.822	0.922 / 0.813	0.252 / 0.105	0.807 / 0.710	0.384 / 0.183
Custom NN (MLP)	0.882 / 0.925	0.949 / 0.838	0.285 / 0.171	0.894 / 0.432	0.432 / 0.245

Comparison vs. Part 1 (tabular only):

Model	Tabular only	+ Graph	Δ
Decision Tree	0.861	0.810	−0.051
XGBoost	0.879	0.865	−0.014
Logistic Regression	0.839	0.813	−0.026
Custom NN (MLP)	0.854	0.838	−0.016

Adding the graph features changes the train/val gap dramatically (train AUC jumps to 0.92-0.96, val AUC drops by 0.01-0.05

Q14 :

Top-10 with graph features:

Rank	Decision Tree	XGBoost	Logistic Regression
1	**graph_degree (0.85)**	**graph_degree (0.25)**	**comments__ContentLicense__count (43.53)**
2	**postHistory__RevisionGUID__count (0.05)**	**postHistory__PostHistoryTypeId__count (0.09)**	**postHistory__ContentLicense__count (9.30)**
3	**postHistory__PostHistoryTypeId__count (0.04)**	**posts__CreationDate__min (0.07)**	**postHistory__UserDisplayName__count (9.28)**

Rank	Decision Tree	XGBoost	Logistic Regression
4	badges__Name__count (0.02)	posts__CreationDate__max (0.06)	posts__OwnerDisplayName__count (8.98)
5	comments__ContentLicense__count (0.01)	badges__Class__count (0.05)	posts__Title__count (8.72)
6	badges__Class__count (0.01)	postHistory__ContentLicense__count (0.04)	badges__TagBased__count (5.01)
7	posts__CreationDate__max (0.005)	wl_K2 (0.04)	badges__Class__count (5.01)
8	comments__Text__count (0.004)	badges__Date__max (0.04)	badges__Date__max (3.26)
9	comments__CreationDate__max (0.003)	comments__ContentLicense__count (0.04)	badges__Date__min (3.12)
10	postHistory__CreationDate__max (0.003)	postHistory__CreationDate__max (0.03)	votes__CreationDate__min (2.75)

Which models used the graph features.

Decision Tree used graph_degree only. XGBoost used graph_degree and also wl_K2 (rank 7). Logistic Regression didn't use any of the 6 graph features we built.

Which graph features were most helpful.

Basically only graph_degree. It gets 85% of the DT importance and 25% of XGB's. wl_K2 made it into XGB's top 10 with a small importance (0.04). wl_K3, wl_K5, and the two graphlet counts (self_comment_count, linked_own_posts_count) didn't show up in any top 10.

Why these results.

graph_degree is just one numeric feature that counts how many things in the database point to the user, so it makes sense that tree models pick up on it. The problem is, with proper temporal filtering it stops being a "magic" feature, it just summarizes past activity, and our tabular features already do that. So the trees end up over-relying on it (DT train AUC 0.925 vs val 0.810) and the val AUC actually goes down compared to Part 1.

WL doesn't help much. The integer color IDs are arbitrary (color 42 isn't similar to color 43), so LR can't use them at all. Trees could split on them but each color class is tiny (about 84k colors across

255k users), so no single split really separates classes. wl_K2 barely makes XGB's top 10 and the higher K versions don't appear at all.

The graphlet counts are too sparse to do anything. self_comment_count is non-zero for around 37k users (~15%) and linked_own_posts_count for only 1.9k users (under 1%). Even if they were predictive for those users, there aren't enough of them to move the needle.

LR doesn't use graph_degree either, even though it's numeric. We think this is because graph_degree is extremely right-skewed (median 1, max 70k), so after standardization it's still mostly bunched up near zero with a long tail. Trees handle that fine with thresholds, but a linear model can't.

Q15 :

What we did and what we learned

We loaded rel-stack and the user-engagement target. The first lesson was the temporal cutoff: every aggregation has to filter to source_date <= target_timestamp. We wrote three manual features in Q1 and an automated join-aggregate algorithm in Q2 (skip PK/FK, drop users.AccountId per Piazza). At depth 2 this gave 51 features. Training four models, XGBoost won at 0.879 val AUC > DT 0.861 > MLP 0.854 > LR 0.839, all above the 0.65 users-only baseline. Boosting wins because of non-linearity and NaN handling; LR loses because most signal is non-linear.

For Part 2 we built three temporally-filtered graphs, computed degree, WL (K=2,3,5), and two graphlet counts. All four models lost 1-5 points of val AUC. Only graph_degree got used by tree models, and it caused overfitting (DT train 0.925 vs val 0.810).

Manual vs automated feature extraction

Manual is quick and easy to validate but doesn't scale. Automated gives breadth and surfaces things we wouldn't pick (timestamp aggregations from postHistory, badges, votes), but produces redundancy and doesn't know which columns are identifiers, which is why Piazza had to clarify the PK/FK skip and AccountId drop.

Graph vs tabular features

With temporal filtering, graph features didn't help and slightly hurt every model. Only graph_degree got used, and it caused overfitting. WL barely appeared (just wl_K2 in XGB top 10) and graphlets didn't appear at all. The cost of building three graphs and the memory pressure didn't pay off over the temporally-correct tabular pipeline.

Q16 :

AI usage

We used Claude across the whole assignment code scaffolding, debugging and drafting the report markdown. We didn't paste anything blindly: we read every cell before running it, ran the outputs, and edited the writeups to match our own voice.

A few concrete validation steps: we counted the Q3 feature totals (16/51/111) by hand to check the algorithm; we noticed the +0.10 AUC jump in our initial Q13 looked suspicious and dug in to find the temporal leakage in graph_degree which forced us to rebuild the graph properly per split; and we caught a few real bugs (a .days vs .dt.days error, an OOM during Q13 training that needed a kernel restart + memory-aware restructure, a float32 overflow from datetime-as-nanoseconds that we fixed by converting to seconds). We also went back through the Piazza Q&A and aligned our pipeline with the TA's clarifications (skip PK/FK in aggregation, drop AccountId entirely, treat badges.Class and badges.TagBased as categorical, build temporal graphs per split, use WL $K \in \{2,3,5\}$).

Estimated AI assistance: about 75–80% of the workflow most of the code and report drafts were AI-generated, but we reviewed, debugged and validated everything before using it. :)